OS Orange

Name - Trisha Vallamkonda Prathap
SRN - PES1UG24CS503
Section - I

Phase - 1

1 A

```
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9
aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B

```
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ find .pes/objects -type
f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase -2

```
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B

```
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ xxd .pes/objects/d5/8213
f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
```

Phase -3

3A

```
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ echo "hello" > file1.txt
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ echo "world" > file2.txt
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ ./pes add file1.txt file
2.txt
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.h
  untracked:  test_objects
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  pes.c
  untracked:  pes.h
  untracked:  test_sequence.sh
  untracked:  index.c
  untracked:  tree.h
  untracked:  commit.c
  untracked:  README.md
  untracked:  tree.c
  untracked:  object.c
  untracked:  test_tree
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  Makefile
```

3B

```
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776686880 6 file1
.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776686897 6 file2
.txt
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$
```

Phase -4

4A

```
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ ./pes log
commit 78afb0205f404f9c262622f1cb6d94a5d8db1d1d80e2f9b044544062ed7b1ff6
Author: TRISHA V P <PES1ug24cs503>
Date:   1776687630

    Add farewell

commit e415815c1770eb2535eac429efd34e6a6094d8a1b3ebe0c81e5bfc441138974c
Author: TRISHA V P <PES1ug24cs503>
Date:   1776687630

    Add world

commit d4d86b9126909a68976c75d10ee4ad42a281ad66c659fabc38b0f7d8744367ac
Author: TRISHA V P <PES1ug24cs503>
Date:   1776687630

    Initial commit
```

4B

```
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/3b/c19934c2ccc37b9582dd42105ca4a1df8e197f51c99a3ef71628231437fdbf
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/74/ba40d3480c9114e80e603252f2df531fdf106287a7288b4895c4150a914f49
.pes/objects/78/afb0205f404f9c262622f1cb6d94a5d8db1d1d80e2f9b044544062ed7b1ff6
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/ae/2edef6f13fc718df618db61577e1db8a70a78c47d027ffed0f397841349d03
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d4/d86b9126909a68976c75d10ee4ad42a281ad66c659fabc38b0f7d8744367ac
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e4/15815c1770eb2535eac429efd34e6a6094d8a1b3ebe0c81e5bfc441138974c
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ed/5ba4eb41194720eb9df8cd9308c507f0a8a5e0406678a69eb5a57d4b6dba84
.pes/refs/heads/main
```

4C

```
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ cat .pes/refs/heads/main

78afb0205f404f9c262622f1cb6d94a5d8db1d1d80e2f9b044544062ed7b1ff6
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Phase -5

5A

```
trisha-vp@trisha-vp-Yoga-7-2-in-1-14IML9:~/PES1UG24CS503-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 1551553caeee... Initial commit

Log after first commit:
commit 1551553caeee917563aed636bccb2fb88bac858f9db9293f530037703a983aa6
Author: TRISHA V P <PES1ug24cs503>
Date:   1776687656

    Initial commit


--- Second Commit ---
Committed: 7944a1c12990... Update file.txt

--- Third Commit ---
Committed: c616cdee24f2... Add farewell

--- Full History ---
commit c616cdee24f26054ec23f47a10da3f728b211a0c31b702eadcdfa4b6f3b85c87
Author: TRISHA V P <PES1ug24cs503>
Date:   1776687656

    Add farewell

commit 7944a1c12990fd22f4f1866aea414b504f84c2460439cef0572755097504126 6
Author: TRISHA V P <PES1ug24cs503>
Date:   1776687656

    Update file.txt

commit 1551553caeee917563aed636bccb2fb88bac858f9db9293f530037703a983aa6
Author: TRISHA V P <PES1ug24cs503>
Date:   1776687656

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
c616cdee24f26054ec23f47a10da3f728b211a0c31b702eadcdfa4b6f3b85c87

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/15/51553caeee917563aed636bccb2fb88bac858f9db9293f530037703a983aa6
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/79/44a1c12990fd22f4f1866aea414b504f84c2460439cef0572755097504126 6
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c6/16cdee24f26054ec23f47a10da3f728b211a0c31b702eadcdfa4b6f3b85c87
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

To implement `pes checkout <branch>`, first read the commit hash stored in `.pes/refs/heads/<branch>`. Using that hash, load the commit object and identify the tree linked to it. Then update HEAD so that it points to the selected branch reference. After that, rebuild the index according to the target branch snapshot. The working directory must also be changed so it matches that tree exactly by adding new

files, updating modified ones, and removing tracked files that are not present in the target branch.

The difficult part is that checkout is more than changing one reference. It has to safely sync branch data, index data, and user files while making sure local uncommitted changes are not lost and the repository does not end up partially updated.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

This conflict can be detected by comparing the current working directory with the index, and then comparing those results with the target branch tree stored in the object database. For every tracked file in the index, first use metadata such as size or modification time for a quick check, and if needed calculate the file hash. If the current file hash does not match the index hash, then the file has local changes.

Next, check whether that same path is different in the target branch tree, such as a changed blob, deleted file, or type difference. If both conditions are true, checkout should stop because switching branches would overwrite local work. It should also refuse if an untracked file exists where the target branch needs to create a file.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In detached HEAD state, `HEAD` stores a commit hash directly instead of pointing to a branch. If commits are made in this state, new commits are still created normally, but no branch pointer moves forward to include them. Because of this, those commits can become hard to find once the user checks out another branch.

To recover them, the user can create a new branch that points to the detached commit hash, then switch to that branch. As long as garbage collection has not removed those unreachable commits, they can still be restored safely.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

A safe way to clean unreachable objects is to use a mark-and-sweep method. In the marking phase, begin from all active references such as files in `.pes/refs/heads/*` and also `HEAD` if it is detached. From these starting points, recursively follow commit links to parent commits and trees, then continue through each tree to the blobs and subtrees it contains.

Use a hash set to store every visited hash, since membership checks are fast and each object is processed only once even if multiple branches share history. In the sweep phase, scan all files inside `.pes/objects/*`. If an object hash is not found in the reachable set, it can be deleted.

For a repository with 100,000 commits and 50 branches, the total visited objects would usually be close to the number of unique commits plus their related trees and blobs. Since branches often share history, it is normally much less than 50 times the commit count, and may range from a few hundred thousand to a few million objects depending on repository contents.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Running garbage collection at the same time as a commit is risky because creating a commit happens in multiple steps. Usually blobs, trees, and the commit object are written first, and only after that does the branch reference get updated.

A race condition can happen if GC scans references before the branch is updated. It may not see the newly created commit as reachable and could delete one of the new objects. Then, when the branch ref is finally updated, it may point to a commit whose required objects are missing.

Git avoids this by using locking during updates, keeping recently unreachable objects for a grace period instead of deleting them immediately, and using reflogs for extra protection. This ensures new objects are not removed while a commit operation is still in progress.